Vivo Fibra Gamer - Developer Guide

AEM Edge Delivery Services with the Universal Editor

Date: October 29, 2026

Adobe

Table of Contents

Vivo Fibra Gamer - Developer Guide 3
- Quick Reference 3
- Architecture Overview 3
- Local Development Setup 4
- Design System 5
- Blocks Reference 6
- Content Modeling Rules (Universal Editor) 8
- Deployment and Code Sync 9
- Site Configuration and Security 9
- Common Development Tasks 9
- Troubleshooting 10
- Environments 10
- Resources 11
- Support Contacts 11

Vivo Fibra Gamer - Developer Guide

This guide is for developers who maintain and extend the Vivo Fibra Gamer site. The site is built with AEM Edge Delivery Services, and content is authored in AEM Sites with the Universal Editor. It recreates the page https://internet.vivo.com.br/ofertas/fibra/gamer/.

Quick Reference

Resource	URL / Value
Config Service organization / site	`harithakrav` / `vivo-gamer-poc` (standard single-site setup)
Code repository	GitHub `harithakrav/vivo-gamer-poc`
Content source	AEM Author `author-p123615-e2372591.adobeaemcloud.com`, site `/content/vivo-gamer-poc`, assets `/content/dam/vivo-gamer-poc`
Development branch	`poc`
Feature preview	https://poc–vivo-gamer-poc–harithakrav.aem.page/
Production preview	https://main–vivo-gamer-poc–harithakrav.aem.page/
Production live	https://main–vivo-gamer-poc–harithakrav.aem.live/
Local development	http://localhost:3000

Important: at the time of writing, all new blocks and fixes are on `poc`. `main` still contains the original boilerplate. Merge `poc` into `main` through a pull request to bring the site to production (see Deployment and Code Sync).

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+) and CSS3 with custom properties; no build step, files are served as they are
- Content authored in AEM Sites with the Universal Editor (the "xwalk" setup), based on `adobe-rnd/aem-boilerplate-xwalk`
- Each block has a `_<block>.json` file with its Universal Editor definitions, models and filters; the combined `component-*.json` files are generated from them

Project Structure

```
├── blocks/                 # 17 blocks (13 custom or customized)
├── models/                 # Universal Editor models for page, section and default
content
├── scripts/
│   ├── aem.js              # Core library (DO NOT MODIFY)
│   ├── scripts.js          # Page decoration, Trusted Types policy, section anchors
│   ├── editor-support.js   # Universal Editor live updates (customized)
│   ├── editor-support-rte.js# Rich text grouping for the Universal Editor
│   ├── dompurify.min.js    # Used by editor-support.js
│   └── delayed.js          # Delayed phase (empty, for analytics/marketing)
├── styles/
│   ├── styles.css          # Critical styles, design tokens, layout, Narrow style
```

```
│   ├── fonts.css            # Roboto 300/400/500/700
│   └── lazy-styles.css      # Non-critical styles (empty)
├── icons/                   # chat, chevron-down, chevron-right, close, whatsapp, wifi
├── fonts/                   # Roboto woff2 files
├── component-definition.json / component-models.json / component-filters.json  #
generated
├── xwalk.json               # enables multi-field rendering (see Content Modeling)
├── head.html                # CSP meta tag, script and style includes
└── favicon.ico              # Vivo favicon (16x16, 1.1 KB)
```

Three-Phase Loading (Eager - Lazy - Delayed)

Phase	Purpose	What loads in this project
Eager	Render the first section as fast as possible (LCP)	`styles.css`, `scripts.js`, the first section (Announcement Banner)
Lazy	Everything else after first paint	Remaining sections and blocks, header (`/nav`), footer (`/footer`), `fonts.css`, `lazy-styles.css`
Delayed	Third-party scripts, about 3 s after load	`delayed.js` (currently empty)

Rules:

- Never add third-party scripts to `scripts.js`; use `delayed.js`.
- The Chat Button loads its iframe only when the drawer is first opened, so it has no effect on page load.

Key Files and How They Connect

File	Role
`scripts/aem.js`	Core library. The project imports `loadHeader`, `loadFooter`, `decorateIcons`, `decorateSections`, `decorateBlocks`, `decorateTemplateAndTheme`, `waitForFirstImage`, `loadSection`, `loadSections`, `loadCSS`, `toClassName`. **Do not modify.**
`scripts/scripts.js`	Entry point. Custom additions: `setupTrustedTypes()` (default Trusted Types policy), `decorateSectionAnchors()` (Anchor ID → section `id`), page language `pt-BR`. Exports `moveInstrumentation` and `decorateMain`.
`scripts/editor-support.js`	Re-renders blocks and sections after Universal Editor edits. Customized so lookups only search inside `main` (see Troubleshooting).
`blocks/fragment/fragment.js`	`loadFragment()` loads `/nav` and `/footer`. Customized to remove duplicate editor instrumentation when the fragment is the page being edited.

Local Development Setup

Prerequisites

- Node.js 20 or newer (the GitHub build workflow uses Node 24)
- AEM CLI: `npm install -g @adobe/aem-cli`
- Access to the GitHub repository and to AEM Author

Setup Steps

```
git clone https://github.com/harithakrav/vivo-gamer-poc.git
cd vivo-gamer-poc
git checkout poc
npm install
aem up
```

The local server runs at http://localhost:3000 with auto-reload. It serves your local code together with content that has been previewed by authors.

Useful Commands

```
npm run lint         # ESLint (JS + Universal Editor models) and Stylelint (CSS)
npm run lint:fix     # auto-fix lint issues
npm run build:json   # regenerate component-definition/models/filters.json from the
_*.json files
```

Run `npm run build:json` after every change to a `_<block>.json` file or to `models/`. Commit the generated files too: the Universal Editor reads the combined files, not the partial ones.

Design System

Breakpoints

The whole project uses one set of breakpoints, written mobile first:

Name	Width	Media query
Mobile	up to 767px	default styles; `(width < 768px)` for mobile-only rules
Tablet	768px to 1023px	`@media (width >= 768px)`
Desktop	1024px and up	`@media (width >= 1024px)`

The header's `isDesktop` check in `header.js` uses `(min-width: 768px)`: the menu collapses into the hamburger **only on mobile**.

Layout

Token	Mobile	Tablet	Desktop
`--content-max-width`	540px	1440px	1440px
`--content-padding`	24px	30px	60px

- All sections, the header and the footer use `max-width: var(--content-max-width)`, `margin: auto` and `padding: 0 var(--content-padding)`. On mobile, content is at most 540px wide, including the 24px padding.
- Sections have **no outer margins**. Authors add vertical space with the Spacer block.
- Full-bleed blocks (Announcement Banner, Hero Banner, CTA Bar) reset their wrapper's `max-width`/`padding` and constrain the inner block instead.

Section Styles

Style (section `Style` field)	Class	Effect
Highlight	`highlight`	Light background, 40px vertical padding (boiler-plate)
Narrow	`narrow`	Content at most 720px (tablet) and 960px (desktop); mobile follows the global 540px

Sections also have an **Anchor ID** field. `decorateSectionAnchors()` turns it into the section's `id` for same-page links (`#ofertas`, `#fibra-gamer`). `scroll-margin-top` keeps anchored sections below the fixed header.

Colors

```
--vivo-purple: #609;            /* links, radios, CTA bar, dots */
--vivo-pink: #ba3566;           /* pricing card CTA */
--vivo-pink-hover: #9e2b55;
--vivo-navy: #050247;           /* hero banner background */
--text-color: #000;
--text-secondary-color: #666;
--border-color: #ccc;
--light-color: #f0f0f0;         /* announcement bar, card details */
--footer-background-color: #f7f7f7;
--link-color: #609;  --link-hover-color: #4a0070;
```

Other colors used in blocks: WhatsApp label `#64d072`, Chat Button label `#fa6423`, inactive carousel dot `#d9c2e6`.

Typography

Token	Mobile	Tablet/Desktop
`--heading-font-size-xxl` (h1)	28px	40px
`--heading-font-size-xl` (h2)	24px	32px
`--body-font-size-m` / `-s` / `-xs`	16 / 14 / 12px	same

Family	Weights	Usage
Roboto (`fonts/roboto-*.woff2`)	300 (body and headings), 400, 500, 700	`--body-font-family`, `--heading-font-family`
Fallback	`roboto-fallback` (Arial, size-adjusted)	Avoids layout shift while fonts load

Header Height

`--nav-height: 76px`. The header is **fixed at the top on all screen sizes**; `header` reserves its height.

Blocks Reference

All blocks below are custom or customized for this project. `cards`, `columns`, `hero` and `fragment` are boilerplate blocks (`fragment` is customized, see above).

Block	Purpose	Universal Editor fields	Behavior and notes
`announcement-banner`	Grey promo strip	Icon, Icon Alt Text, Text (rich text, bold lead-in)	Full-bleed grey background
`pricing-cards`	Heading plus plan cards (container)	Block: Eyebrow, Heading, Heading Type, 4 labels ("Mais detalhes", apps heading, "Conferir apps", pop-up heading). Item **Pricing Card**: Speed, Speed Description, Option 1 / Option 2 (Card Title, Label, Price, Details, Checkout URL, Button Label)	Radio options switch card title, price and checkout link. "Mais detalhes" expands the apps preview (first 3 logos + "+N"). "Conferir apps" opens a right-side drawer (`<dialog>`, 630px on tablet/desktop). Carousel with indicator dots when cards overflow.
`included-apps`	Apps shown in the pricing cards (container)	Item **App**: App Logo, App Logo Alt Text, App Name, App Description	Must be in the **same section** as Pricing Cards (first match on the page otherwise). Hidden on the page; shown as a dashed list in the editor. Pricing Cards reads its rows, so `included-apps.js` must keep the rows intact.
`link-list`	Full-width links with chevrons (container)	Item **Link**: Link, Text, Title	PDFs and external links open in a new tab. Empty items stay visible in the editor as placeholders.
`hero-banner`	Dark campaign banner (container)	Block: Campaign Image, Alt, Eyebrow, Heading, Heading Type. Item **Benefit**: Icon, Icon Alt Text, Benefit Text	Mobile: image first, stacked benefits. Tablet/desktop: two columns, benefits grid.
`partner-promo`	Partner banner	Partner Logo, Alt, Description (rich text), CTA Link, CTA Label	Column on mobile, row from 768px
`legal-disclaimer`	Small print	Legal Text (rich text)	14px grey text
`cta-bar`	Sticky call bar	Label, Phone Number, Link (optional)	Makes its **section** sticky at the bottom (`.cta-bar-container`), so it must be the last section. Builds `tel:` from the phone digits.
`whatsapp`	Floating WhatsApp icon	WhatsApp Number, Pre-filled Message, Accessible Label, Hover Text	Opens `https://wa.me/<digits>?text=...` in a new tab. The green label expands on hover; line breaks come from Enter in Hover Text (`readLines()`).
`chat-button`	Floating chat icon above WhatsApp	Chat URL, Hover Text, Accessible Label	Orange label on hover. Click opens a right-side drawer with the chat page in an iframe (loaded on first open). Imports `readLines` from `whatsapp.js`.
`spacer`	Vertical space	Height, Mobile Height (selects → block classes `height-N`, `mobile-N`)	0 to 120px. Striped placeholder in the editor.
`header`	Loads `/nav`	Authored as a page: section 1 = logo images, section 2 = Text with a bulleted list of links	Fixed. Hamburger on the right on mobile; open menu is a dropdown panel with divider and dimmed backdrop (tap to close). Links close the menu.
`footer`	Loads `/footer`	Authored as a page: copyright Text, privacy link Text, 5G logo Image	Images on their own paragraph get `footer-logo` and sit on the right

Pricing Card Widths

Cards are sized with a container query on the card list (`container: pricing-cards-list / inline-size`):

- Minimum card width **235px**. As many cards per view as fit share the width: 2 per view from 482px of list width, 3 from 729px, 4 from 976px, 5 from 1223px.
- One full-width card per view when two don't fit (most phones).
- Tablet and desktop: a card is at most **310px**.

- The dots (`buildDots()`) are shown only while `scrollWidth > clientWidth`; a `ResizeObserver` updates them live.

Expected Markup (example: Pricing Card item)

AEM renders each model field as a cell; grouped fields (`option1_*`) share one cell:

```
<div>                                  <!-- one Pricing Card row -->
  <div><p>700 Mega</p><p>de internet com Wi-Fi grátis</p></div>
  <div><p>Vivo Fibra Gamer</p><p>Fibra Gamer</p><p>R$ 170/mês</p>
       <p>+ Exitlag e ponto cabeado incluso</p><p><a href="...">Consultar Fibra</a></p></
div>
  <div><p>Vivo Fibra</p><p>Vivo Fibra</p><p>R$ 150/mês</p><p><a href="...">Consultar
Fibra</a></p></div>
</div>
```

`pricing-cards.js` tells rows apart by cell count: 1 cell = heading/labels, 2 cells = apps authored in the old structure, 3 or more = cards. Empty grouped fields are left out of the cell, so options are parsed by order: title, label, price, then details if present, then the link.

Preserving Universal Editor Instrumentation

When a block rebuilds its DOM, move the `data-aue-*` attributes from the authored row to the new element with `moveInstrumentation(from, to)` (see `pricing-cards.js`, `link-list.js`, `hero-banner.js`). Two rules learned on this project:

- Never put an item's instrumentation inside another item's element. The editor builds its content tree from the DOM, so an app nested inside a card was shown as a child of that card and could not be deleted.
- Keep incomplete items selectable in the editor (check `row.hasAttribute('data-aue-resource')`). Otherwise a newly added, still empty item disappears and can't be edited.

Content Modeling Rules (Universal Editor)

- **Field collapse:** names ending in `Text`, `Title`, `Type`, `Alt`, `MimeType` are merged into the field without the suffix (`link` + `linkText` → `<a>`). A field like `hoverText` without a `hover` field is dropped; use names like `hoverLabel`. The lint rule `xwalk/no-orphan-collapsible-fields` catches this.
- **Element grouping:** `group_field` names render in one cell (`heading_eyebrow`, `heading_title`, `heading_titleType` → `<p>` + `<h1>`).
- **At most 4 cells per model** (`xwalk/max-cells`). Use grouping to stay within the limit.
- **Only one level of items:** a block can contain items, but items can't contain items. That is why Included Apps is a separate block and not part of Pricing Cards.
- **Multi-fields / composite (`container`, `multi: true`) fields** only render when the multi-field flag in `xwalk.json` is active **for the code branch AEM uses to render content**. Content is rendered with `main`, so these fields came through as empty cells while `main` didn't have `xwalk.json`. The current blocks don't use multi-fields.
- **Text component:** the boilerplate Text component intentionally has no model. Text is edited inline, so an empty properties panel is expected.
- Run `npm run build:json` and `npm run lint` after every model change.

Deployment and Code Sync

Workflow

1. Work on `poc` (or a feature branch), test locally, run `npm run lint`.
2. Push. AEM Code Sync updates `https://<branch>--vivo-gamer-poc--harithakrav.aem.page/`.
3. Open a pull request to `main` with before/after preview URLs (see `.github/pull_request_template.md`). The `build` GitHub Action runs `npm ci` and `npm run lint`.
4. After review, merge. `main` is production: the live site and the Universal Editor use it immediately.

Recommended `main` ruleset ("Protect main (production)"): require a pull request, require the `build` status check, block force pushes and deletions; optionally require CODEOWNERS review.

Code Sync Only Copies Changed Files

AEM Code Sync copies only the files that change in a push. Files that were never changed after the initial commit (`scripts/editor-support-rte.js`, `scripts/dompurify.min.js`, `scripts/delayed.js`, `xwalk.json`) were missing from the code bus (404) until a commit changed them. After setting up a branch or a new site, check that every file is served:

```
B=https://<branch>--vivo-gamer-poc--harithakrav.aem.page
for f in $(git ls-files scripts styles blocks icons fonts '*.json' '*.html' '*.ico' | grep
-v '/_'); do
  c=$(curl -s -o /dev/null -w '%{http_code}' "$B/$f"); [ "$c" != 200 ] && echo "$c $f"
done
```

`_*.json` files are intentionally not served (`.hlxignore`). To push a missing file, make a no-op change to it (for example a comment) and push again.

Site Configuration and Security

- **Content-Security-Policy:** the delivered CSP includes `require-trusted-types-for 'script'` and `frame-src 'self' https:`, which come from outside the repository: `head.html` only defines `script-src`, `base-uri` and `object-src`. Under Trusted Types, the boilerplate's `innerHTML` (fragment loading) and script URL assignments throw. `setupTrustedTypes()` in `scripts.js` registers a default pass-through policy before decoration. Keep it as long as that CSP directive is in place.
- **External iframe:** the Chat Button loads `https://plataforma.portal.vivo.com.br/chat/fibra/`. If that site stops allowing embedding, the drawer opens empty; fix that on the chat service side.
- **Edge Delivery file limits:** SVG 40 KB, favicon 16 KB, images 20 MB, PDF 20 MB. SVGs that embed raster images can exceed 40 KB: downscale the embedded image (the Abril News logo was reduced from 42.6 KB to 14 KB).
- No secrets in the repository; everything is public client-side code.

Common Development Tasks

Add a New Block

1. Create `blocks/<name>/<name>.js` (`export default function decorate(block)`), `<name>.css` (selectors scoped to `.<name>`), and `_<name>.json` (definition, model, filter).
2. Add the block id to the section filter in `models/_section.json`.
3. Run `npm run build:json` and `npm run lint`.

4. Test with a local `.plain.html` page using the same markup AEM renders, then on the branch preview and in the Universal Editor.

Add a Field to an Existing Block

Fields map to cells by position. Add new fields at the **end** of the model and handle missing cells, so content authored before the change still renders.

Add Analytics or Marketing Tags

Add them to `scripts/delayed.js` only.

Troubleshooting

Symptom	Cause	Fix
A newly added Text component can't be selected and is missing from the content tree	`editor-support-rte.js` was 404 on the code bus, so `editor-support.js` failed to load	Make sure all scripts are synced (see Code Sync)
New components appear only after reloading the editor	`dompurify.min.js` missing (404), or the edit was applied to the header/footer copy on the nav/footer page	Sync the files; `editor-support.js` looks only inside `main`
Nav or footer page: added Text is not in the content tree	Same content rendered twice (page + header/footer) with the same instrumentation	`fragment.js` removes duplicate instrumentation from the fragment copy
Pricing card shows only the speed	Composite multi-fields rendered as empty cells (`xwalk.json` not active for `main`)	Current model uses option groups and Included Apps items instead
Header and footer don't load on aem.page ("This document requires 'TrustedHTML' assignment")	CSP `require-trusted-types-for 'script'`	Default Trusted Types policy in `scripts.js`
Publishing fails: asset exceeds the 40 KB size limit	SVG with an embedded raster image	Downscale the embedded image or use PNG
A new Link List item disappears in the editor	Rows without a link were dropped during decoration	Placeholder for instrumented empty items
Included Apps shown under the first Pricing Card and can't be deleted	App instrumentation was nested inside a card	Apps live in the separate Included Apps block
Page title shows "AEM Boilerplate"	Page Title not set in page properties	Set Title (and Description) in page properties and publish
Old favicon or styles	Browser cache	Hard refresh (Ctrl+Shift+R); preview cache is 60 s

Environments

Environment	URL	Purpose
Local	http://localhost:3000	Development
Feature branch	https://poc-vivo-gamer-poc-harithakrav.aem.page	Testing on `poc`

Environment	URL	Purpose
Preview	https://main–vivo-gamer-poc–harithakrav.aem.page	Production staging
Live	https://main–vivo-gamer-poc–harithakrav.aem.live	Production

Resources

Resource	URL
Edge Delivery Services documentation	https://www.aem.live/docs/
Content modeling for AEM authoring	https://www.aem.live/developer/component-model-definitions
Universal Editor blocks	https://www.aem.live/developer/universal-editor-blocks
Keeping it 100 (performance)	https://www.aem.live/developer/keeping-it-100
Limits	https://www.aem.live/docs/limits
Boilerplate (xwalk)	https://github.com/adobe-rnd/aem-boilerplate-xwalk

Support Contacts

Role	Name	Contact
Project owner		
Lead developer		
AEM administrator		